# uGDX School of Technology

Mini Project Report on

**“VaaniSetu: An AI-Powered Multilingual Document Reader using Vertex AI Gemini, Cloud Vision, Cloud Text-to-Speech and Cloud Run”**

Submitted in partial fulfillment of the requirements
for the course

**Course: Cloud Application Development (CAP 303)**
**Class: TY B. Tech (Semester V)**

By

Mohak Mandwani 2406506

Supervisor:
Dr. Kunal Meher
(Assistant Professor)

AY: 2026-2027

# uGDX School of Technology

## CERTIFICATE

This is to certify that the project entitled "**VaaniSetu: An AI-Powered Multilingual Document Reader using Vertex AI Gemini, Cloud Vision, Cloud Text-to-Speech and Cloud Run"** is a bonafide work of

**Mohak Mandwani 2406506**

submitted in partial fulfilment of the requirement for the course "**Cloud Application Development**" of Third Year of Bachelor degree in Technology (CS, AI & ML) during academic year 2026-27.

Supervisor/Guide
(Dr. Kunal Meher)

External Examiner

# Table of Contents


1. **Introduction**
2. **Block Diagram**
3. **Technologies used**
    a. **Development Technologies**
    b. **Cloud Services (Deployment)**
4. **Code**
5. **Screenshots**
6. **Conclusion**

# 1. Introduction

Everyday life in India runs on paper: doctors' prescriptions, electricity bills, college notices, bank letters and government forms. Many people cannot read these documents comfortably: people with low vision, elderly citizens, people with limited literacy, and migrants who read a different language from the one the document is printed in. Missing a due date on a bill or misreading a dosage on a prescription has real consequences.

**VaaniSetu** (Hindi for "bridge of voice") is a serverless cloud application that solves this problem. The user simply photographs a document with a phone or uploads an image. The application reads the document, understands what it means, explains it in simple words in the language the user chooses, and **speaks the explanation aloud**. It also lists the key facts and the actions the reader must take, such as paying a bill before a deadline.

## 1.1 Objectives

- Build a fully serverless web application on Google Cloud Platform (GCP) that needs no servers to manage.
- Combine at least three cloud services: this project integrates **eight core Google Cloud services** (Cloud Storage, Cloud Run functions, API Gateway, Cloud Vision, Vertex AI, Cloud Text-to-Speech, Firestore and Cloud Storage website hosting) plus IAM and Cloud Logging.
- Use generative AI (Gemini on Vertex AI) to read and explain documents in seven languages, and neural text-to-speech (Cloud Text-to-Speech) to speak the explanation.
- Keep the system secure (private storage, signed URLs, least-privilege service accounts) and cost-efficient (pay-per-use, scale to zero, no idle cost).

## 1.2 Key Features

- **Photo to speech in about 11 seconds**: upload or capture a document photo and hear it explained.
- **Seven output languages**: English, Hindi, Spanish, French, German, Japanese and Arabic, each with a natural neural voice. The source document can be in any language (for example, a Hindi bill explained in English).
- **Structured understanding**: document type, title, plain-language summary, key points and action items with deadlines, plus the full transcribed text.
- **Smart gatekeeping**: Cloud Vision checks that the photo really contains text before the Gemini model is called.
- **History**: every processed document is stored in Firestore and can be replayed.
- **Accessible design**: large type, high contrast, keyboard navigation and a live progress tracker that shows each Google Cloud service as it runs.

# 2. Block Diagram

Figure 2.1 shows the architecture of VaaniSetu. All components run in the Google Cloud Mumbai region (asia-south1), except API Gateway, which is not offered in Mumbai and runs in the nearest supported region, Tokyo (asia-northeast1).

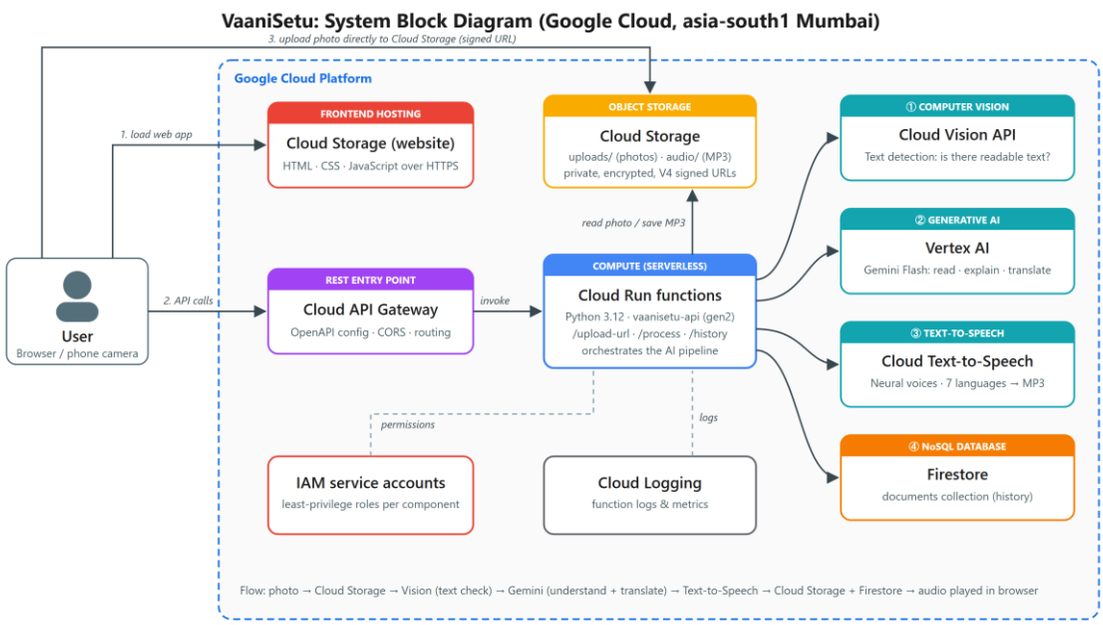



*Figure 2.1: System block diagram of VaaniSetu on Google Cloud*

## 2.1 Workflow

1. The user opens the web application, a static website served over HTTPS from **Cloud Storage**.
2. The browser resizes the photo and calls **POST /upload-url** on **Google Cloud API Gateway**. The **Cloud Run function** returns a short-lived V4 signed URL.
3. The browser uploads the photo **directly to a private Cloud Storage bucket** using that URL, so large files never pass through the API.
4. The browser calls **POST /process**. The function asks **Cloud Vision** (text detection) whether the image contains readable text. Photos without text are rejected at this point.
5. The function sends the image to **Gemini 3.5 Flash-Lite on Vertex AI**, which transcribes the document and returns JSON: document type, title, a spoken summary in the chosen language, key points and action items.
6. **Cloud Text-to-Speech** converts the summary to speech with a neural voice. The MP3 file is saved to Cloud Storage.
7. The result is stored in **Firestore** and returned to the browser with signed links to the image and audio, and the audio plays automatically.
8. **IAM service accounts** limit each component to exactly the roles it needs, and **Cloud Logging** records every request.

# 3. Technologies Used

## 3.1 Development Technologies

| Technology | Purpose in VaaniSetu |
|---|---|
| Python 3.12 | Backend business logic running as a Cloud Run function (gen2). |
| Google Cloud client libraries | google-cloud-storage, -vision, -texttospeech and -firestore, plus google-genai for Gemini on Vertex AI. |
| Functions Framework | Exposes the Python handler as an HTTP endpoint on Cloud Run. |
| HTML5 | Page structure: upload area, language selector, results, history and "how it works" sections, with semantic, accessible markup. |
| CSS3 | Styling: responsive grid layout, custom design tokens, animated pipeline tracker and waveform, mobile layout and reduced-motion support. |
| JavaScript (ES2020) | Dynamic features: drag-and-drop and camera upload, in-browser image resizing (Canvas API), Fetch API calls, live pipeline status, audio playback and history cards. |
| JSON and OpenAPI 2.0 | Data format between browser, API and AI model; API Gateway configuration. |
| gcloud CLI and Bash | Repeatable provisioning and deployment from Cloud Shell (infra/deploy.sh). |
| Git and GitHub | Version control with small, atomic commits. |

## 3.2 Cloud Services (Deployment)

The following Google Cloud services are used. All of them are serverless or fully managed, so the application has no servers to patch and costs nothing while idle.

| Google Cloud Service | Role in VaaniSetu | Configuration |
|---|---|---|
| Cloud Storage | Stores uploaded document photos (uploads/) and generated speech (audio/). | Private bucket in asia-south1, public access prevention enforced, uniform access, CORS for browser upload, uploads deleted after 30 days, V4 signed URLs. |
| Cloud Run functions | Serverless backend that runs the whole AI pipeline. | vaanisetu-api (gen2), Python 3.12, 512 MB, 60 s timeout, 0 to 3 instances. |
| API Gateway | Public HTTPS entry point for the web app. | OpenAPI config with POST /upload-url, POST /process, GET /history and CORS; gateway in asia-northeast1; calls the private function with its own service account. |
| Vertex AI (Gemini) | Generative AI that reads, understands, simplifies and translates the document. | Gemini 3.5 Flash-Lite (multimodal), global endpoint, JSON output, temperature 0.2, thinking off. |
| Cloud Vision API | Computer-vision gate: detects printed text before calling Gemini. | Document text detection reading the image directly from Cloud Storage; at least 2 lines required. |
| Cloud Text-to-Speech | Neural text-to-speech for the explanation. | Neural2 / WaveNet voices for en-IN, hi-IN, es-ES, fr-FR, de-DE, ja-JP and ar-XA; MP3 output. |
| Firestore | Stores the history of processed documents. | Native mode, asia-south1, free tier; collection documents ordered by createdAt. |
| Cloud Storage website | Hosts the static frontend over HTTPS. | Separate public-read bucket vaanisetu-web-<project> with index.html as main page. |
| IAM | Security: grants each component only the permissions it needs. | Service accounts vaanisetu-fn, vaanisetu-gateway and vaanisetu-build with least-privilege roles. |
| Cloud Logging | Monitoring: request logs, errors and latency. | Function logs and metrics; budget alert of 400 INR at 50%, 90% and 100%. |

# 4. Code

The complete source code is in the project repository (https://github.com/17mohak/vaanisetu). It is organised into backend/ (Cloud Run function), frontend/ (web app), infra/ (deployment script, API Gateway config, sample generator) and report/. The most important parts are shown below.

## 4.1 Cloud Run function: request routing

**backend/main.py: api**

```
@functions_framework.http
def api(request):
    if request.method == "OPTIONS":
        return ("", 204, CORS)
    path = request.path
    try:
        body = (request.get_json(silent=True) or {}) if request.method == "POST" else {}
        if path.endswith("/upload-url") and request.method == "POST":
            return create_upload_url(body)
        if path.endswith("/process") and request.method == "POST":
            return process_document(body)
        if path.endswith("/history") and request.method == "GET":
            return get_history()
        return respond(404, {"error": "Route not found"})
    except ValueError as e:
        return respond(400, {"error": str(e)})
    except Exception as e:  # logged to Cloud Logging
        print("ERROR", repr(e))
        return respond(500, {"error": "Processing failed. Please try again."})
```

## 4.2 AI pipeline (Vision → Gemini → Text-to-Speech → Firestore)

**backend/main.py: process_document**

```
def process_document(body):
    key = body.get("key", "")
    lang = body.get("language", "en")
    if not re.fullmatch(r"uploads/[0-9a-f]{12}\.(jpg|png)", key):
        raise ValueError("Invalid document key")
    if lang not in LANGUAGES:
        raise ValueError("Unsupported language")
    doc_id = key.split("/")[1].split(".")[0]
    timings = {}

    blob = bucket.blob(key)
    image_bytes = blob.download_as_bytes()
    if len(image_bytes) > MAX_IMAGE_BYTES:
        raise ValueError("Image is larger than 5 MB")

    # 1. Cloud Vision - verify the photo actually contains readable text
    t = time.time()
    result = vision_client.document_text_detection(

image=vision.Image(source=vision.ImageSource(gcs_image_uri=f"gs://{BUCKET}/{key}")))
    if result.error.message:
        raise RuntimeError(result.error.message)
    text = result.full_text_annotation.text or ""
    lines = [l for l in text.split("\n") if l.strip()]
    blocks = [b for p in result.full_text_annotation.pages for b in p.blocks]
    timings["vision"] = round(time.time() - t, 2)
    if len(lines) < 2:
        raise ValueError("No readable text found. Please upload a clearer photo of a
document.")
    ocr_confidence = round(100 * sum(b.confidence for b in blocks) / max(len(blocks),
1), 1)

    # 2. Vertex AI (Gemini) - read, understand, simplify and translate
    t = time.time()
    analysis = analyse_with_gemini(image_bytes, "image/png" if key.endswith("png") else
"image/jpeg", lang)
    timings["gemini"] = round(time.time() - t, 2)
```

```
    # 3. Cloud Text-to-Speech - convert the simplified explanation into speech
    t = time.time()
    audio = synthesize(analysis["spoken_summary"][:2900], lang)
    audio_key = f"audio/{doc_id}-{lang}.mp3"
    bucket.blob(audio_key).upload_from_string(audio, content_type="audio/mpeg")
    timings["tts"] = round(time.time() - t, 2)

    # 4. Firestore - persist the result
    record = {
        "createdAt": int(time.time() * 1000),
        "docId": doc_id,
        "imageKey": key,
        "audioKey": audio_key,
        "language": lang,
        "documentType": analysis["document_type"],
        "title": analysis["title"],
        "sourceLanguage": analysis["source_language"],
        "extractedText": analysis["extracted_text"],
        "summary": analysis["spoken_summary"],
        "keyPoints": analysis["key_points"],
        "actionItems": analysis["action_items"],
        "ocrLines": len(lines),
        "ocrConfidence": ocr_confidence,
        "timings": timings,
    }
    db.collection(COLLECTION).document(f"{doc_id}-{lang}").set(record)
    return respond(200, with_urls(record))
```

## 4.3 Gemini prompt and Vertex AI call

**backend/main.py: prompt and analyse_with_gemini**

```
PROMPT = """You are VaaniSetu, an assistant that helps visually impaired and low-
literacy people understand documents.
Look at the photographed document and reply with ONLY a JSON object with these keys:
"document_type": short category in English (e.g. "Medical Prescription", "Electricity
Bill", "Government Notice", "Bank Letter", "Exam Timetable", "Form", "Other"),
"title": a short title for the document in English (max 8 words),
"source_language": the language the document is written in (English name),
"extracted_text": the full text of the document, transcribed faithfully, preserving line
breaks,
"spoken_summary": a warm, simple explanation of what the document says and what the
reader must do, written in {language} for being READ ALOUD (plain sentences, no bullet
symbols, no markdown, under 900 characters; say amounts and dates naturally, and read
phone or ID numbers digit by digit),
"key_points": array of 3-5 short key facts in {language}, written with normal digits
(e.g. 2,170.50, 10 October 2026),
"action_items": array of 0-3 things the reader must do (with deadlines if any) in
{language}, written with normal digits.
Never invent information that is not in the document."""



def analyse_with_gemini(image_bytes, mime, lang):
    contents = [types.Part.from_bytes(data=image_bytes, mime_type=mime),
                PROMPT.replace("{language}", LANGUAGES[lang]["name"])]
    base = dict(temperature=0.2, max_output_tokens=4096,
response_mime_type="application/json")
    try:
        # Thinking off: transcription and summarising need no long reasoning, and
        # thinking tokens add latency and are billed as output tokens.
        response = gemini.models.generate_content(
            model=MODEL_ID, contents=contents,
            config=types.GenerateContentConfig(
                **base, thinking_config=types.ThinkingConfig(thinking_budget=0)))
    except Exception as e:  # model does not allow turning thinking off
        print("Gemini thinking_budget fallback", repr(e))
        response = gemini.models.generate_content(
            model=MODEL_ID, contents=contents,
config=types.GenerateContentConfig(**base))
    text = response.text or ""
    match = re.search(r"\{.*\}", text, re.S)
    data = json.loads(match.group(0) if match else text)
    for field in ("document_type", "title", "source_language", "extracted_text",
"spoken_summary"):
        data[field] = str(data.get(field) or "")
```

```
    data["key_points"] = [str(x) for x in data.get("key_points") or []][:5]
    data["action_items"] = [str(x) for x in data.get("action_items") or []][:3]
    if not data["spoken_summary"]:
        raise ValueError("Could not understand this document")
    return data
```

## 4.4 Signed Cloud Storage URLs

**backend/main.py: signed_url**

```
def signed_url(key, method, minutes, content_type=None):
    """V4 signed URL. Cloud Run has no private key, so signing goes through the
    IAM Credentials signBlob API using the function's own service account."""
    if not credentials.valid:
        credentials.refresh(google_requests.Request())
    return bucket.blob(key).generate_signed_url(
        version="v4", method=method, expiration=datetime.timedelta(minutes=minutes),
        content_type=content_type,
service_account_email=credentials.service_account_email,
        access_token=credentials.token,
    )
```

## 4.5 Least-privilege service accounts

**infra/deploy.sh: service accounts and roles**

```
echo "== Service accounts and roles"
for sa in vaanisetu-fn:"VaaniSetu function" vaanisetu-gateway:"VaaniSetu API Gateway"
vaanisetu-build:"VaaniSetu function builds"; do
  id=${sa%%:*}; name=${sa#*:}
  gcloud iam service-accounts describe "$id@$PROJECT.iam.gserviceaccount.com" >/dev/null
2>&1 || \
    gcloud iam service-accounts create "$id" --display-name="$name"
done
sleep 5
for role in roles/datastore.user roles/aiplatform.user
roles/serviceusage.serviceUsageConsumer roles/logging.logWriter; do
  gcloud projects add-iam-policy-binding "$PROJECT" --member="serviceAccount:$FN_SA" --
role="$role" \
    --condition=None >/dev/null
done
for role in roles/storage.objectCreator roles/storage.objectViewer; do
  gcloud storage buckets add-iam-policy-binding "gs://$MEDIA" --
member="serviceAccount:$FN_SA" --role="$role" >/dev/null
done
gcloud projects add-iam-policy-binding "$PROJECT" --member="serviceAccount:$BUILD_SA" \
  --role=roles/cloudbuild.builds.builder --condition=None >/dev/null
# lets the function sign Cloud Storage URLs with its own identity (IAM signBlob)
gcloud iam service-accounts add-iam-policy-binding "$FN_SA" --
member="serviceAccount:$FN_SA" \
  --role=roles/iam.serviceAccountTokenCreator >/dev/null
```

## 4.6 API Gateway configuration (OpenAPI)

**infra/openapi.template.yaml (excerpt)**

```
swagger: "2.0"
info:
  title: vaanisetu-api
  description: VaaniSetu AI document reader API
  version: "1.0.0"
schemes:
  - https
produces:
  - application/json
x-google-backend:
  address: __FUNCTION_URL__
  path_translation: APPEND_PATH_TO_ADDRESS
  deadline: 60.0
paths:
  /upload-url:
    post:
      operationId: uploadUrl
      summary: Get a signed Cloud Storage upload URL
      responses:
        "200": { description: Signed URL issued }
    options:
      operationId: uploadUrlCors
      responses:
        "204": { description: CORS preflight }
```

## 4.7 Frontend: upload and processing

**frontend/app.js: direct-to-Cloud-Storage upload and pipeline call**

```
$("go").addEventListener("click", async () => {
  if (!imageBlob) return;
  hideError();
  $("go").disabled = true;
  $("result").hidden = true;
  $("empty").hidden = false;
  $("empty").classList.add("busy");
  $("empty").querySelector("p").textContent = "Reading your document…";
  let ticker;
  try {
    setStep("upload");
    const up = await api("/upload-url", { contentType: "image/jpeg" });
    const put = await fetch(up.uploadUrl, { method: "PUT", headers: { "Content-Type":
"image/jpeg" }, body: imageBlob });
    if (!put.ok) throw new Error("Upload to Cloud Storage failed");

    // The server runs the remaining stages in one call; animate them while we wait.
    setStep("vision");
    ticker = [setTimeout(() => setStep("gemini"), 1200), setTimeout(() =>
setStep("tts"), 9000)];
    const result = await api("/process", { key: up.key, language: selectedLang });
    ticker.forEach(clearTimeout);
    setStep("all");
    showResult(result, true);
    loadHistory();
  } catch (err) {
    (ticker || []).forEach(clearTimeout);
    setStep("none");
    showError(err.message);
    $("empty").querySelector("p").textContent = "Your explanation will appear and play
here.";
  } finally {
    $("go").disabled = false;
    $("empty").classList.remove("busy");
  }
});
```

# 5. Screenshots

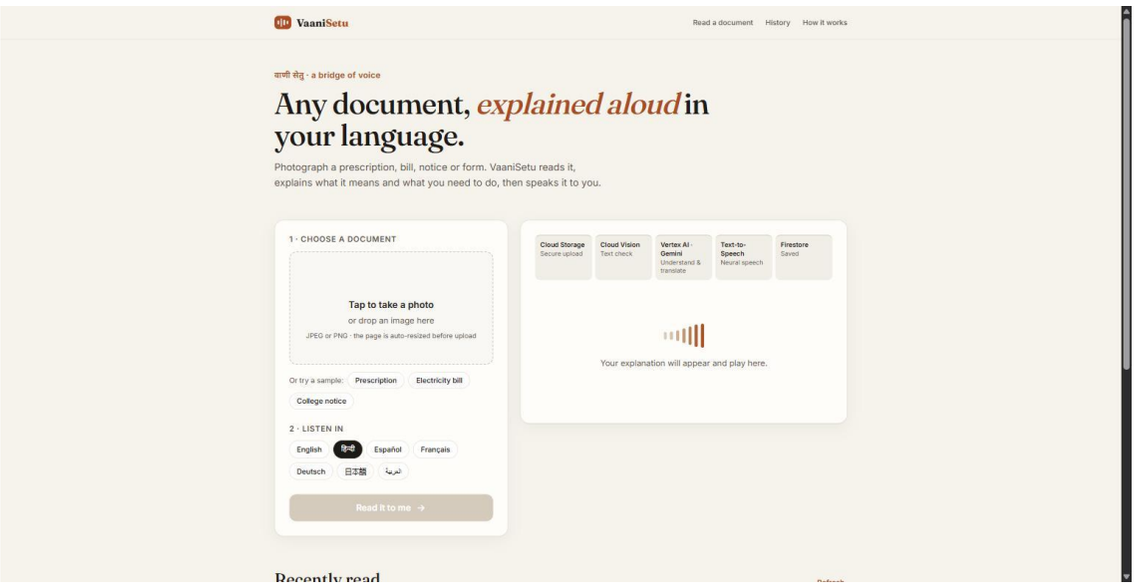



*Figure 5.1: VaaniSetu home page (hosted on Cloud Storage)*

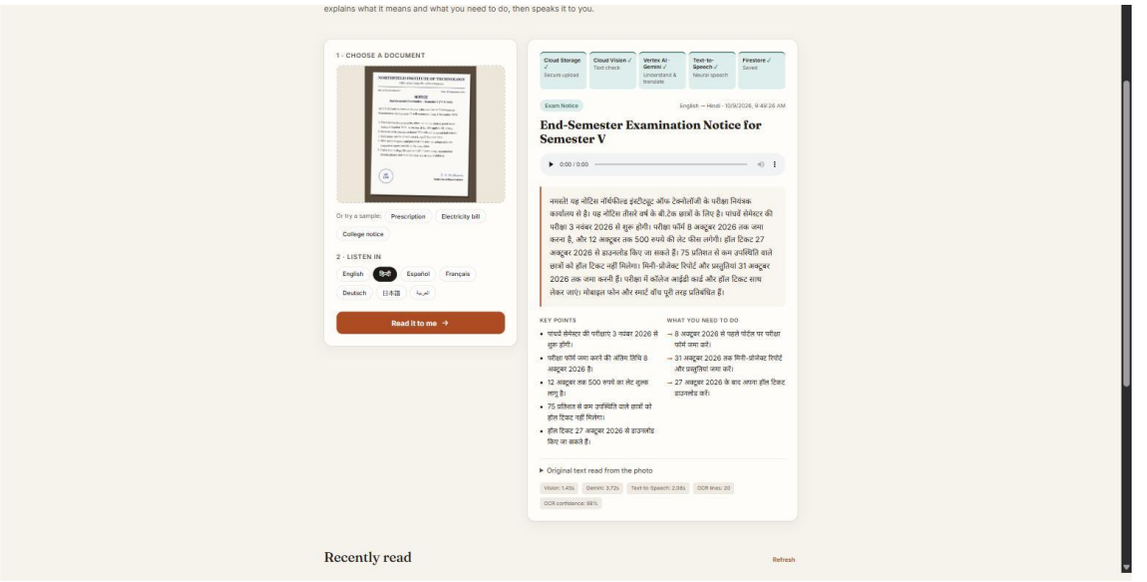



*Figure 5.2: English college notice explained and spoken in Hindi; the pipeline tracker shows every Google Cloud service completed*

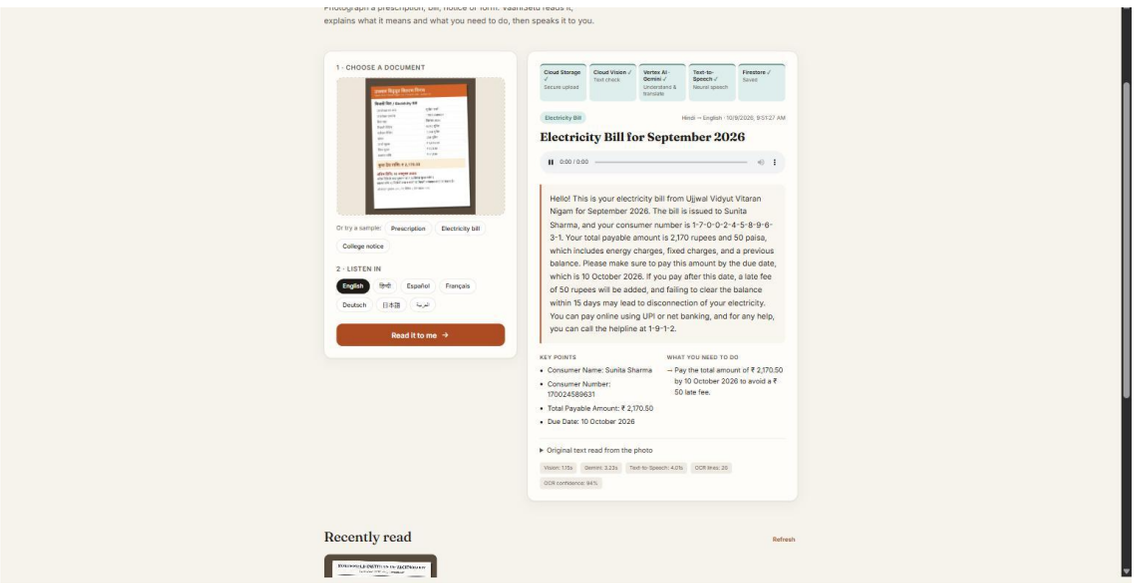



*Figure 5.3: Hindi electricity bill explained in English, with key points, action items and per-service timings*

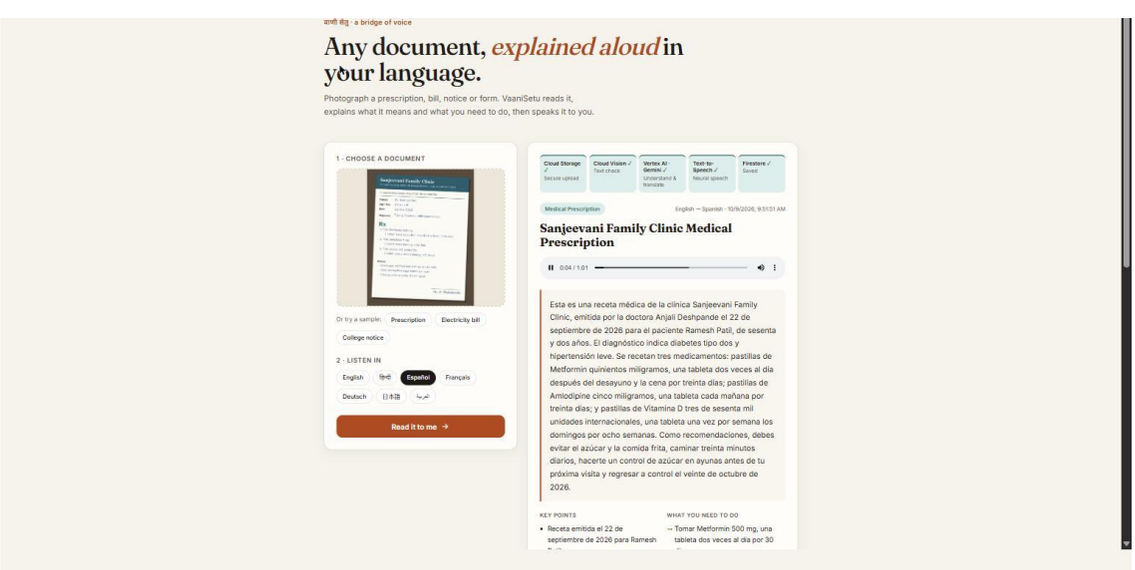



*Figure 5.4: Prescription explained in Spanish with the audio playing*

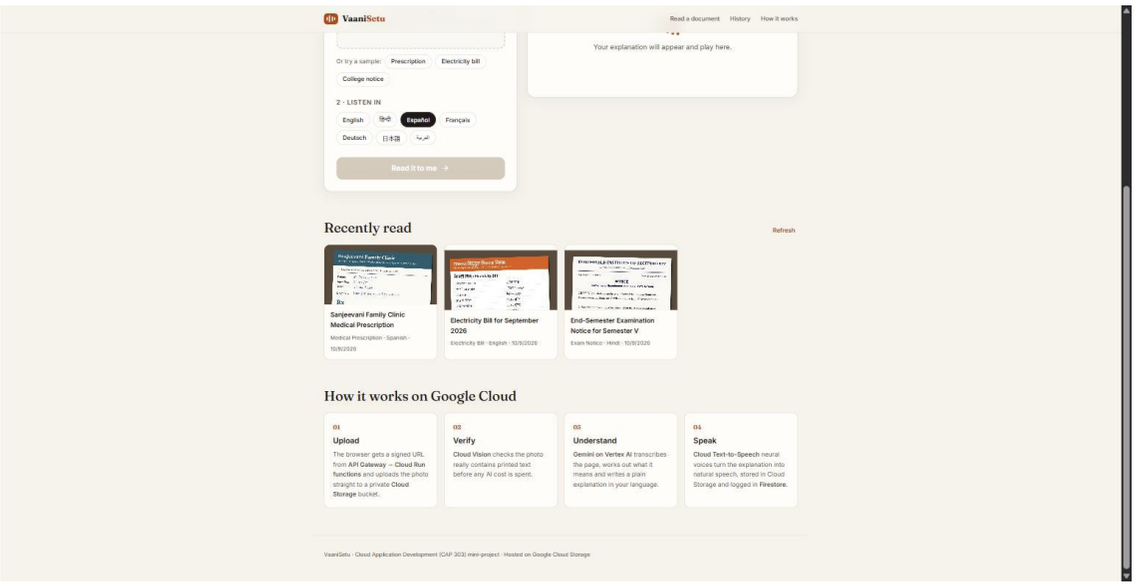



*Figure 5.5: History of processed documents loaded from Firestore*

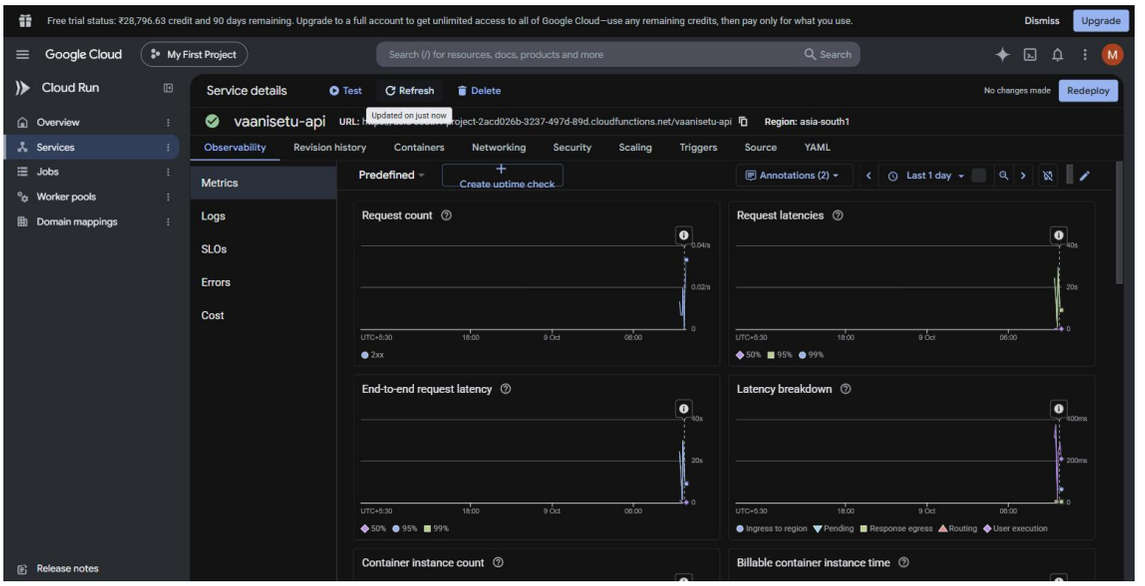



*Figure 5.6: Cloud Run function vaanisetu-api with request metrics*

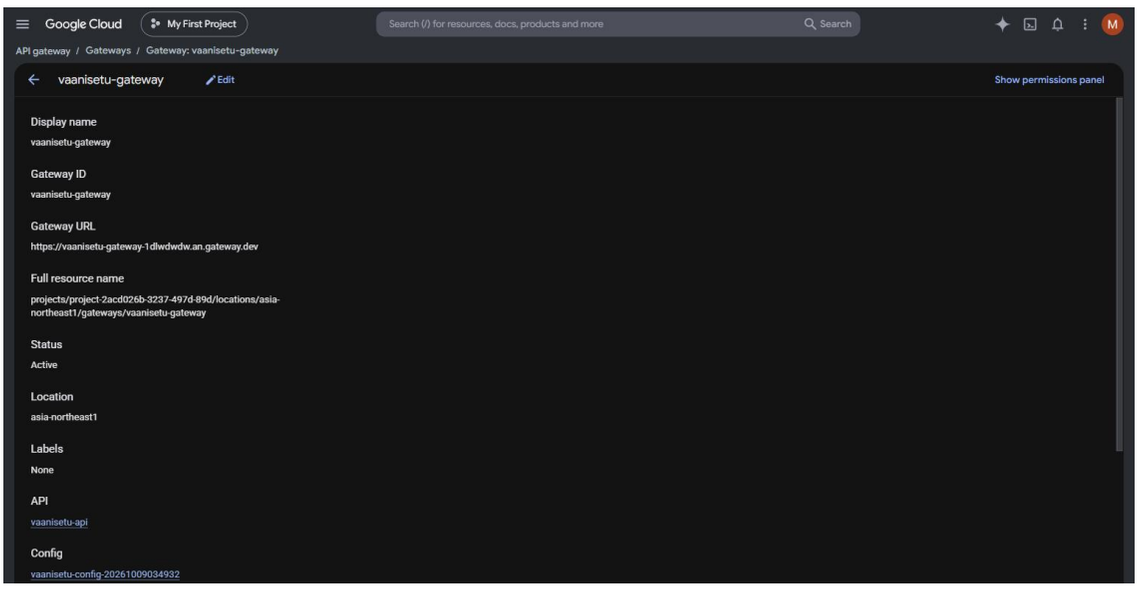



*Figure 5.7: API Gateway vaanisetu-gateway (active, asia-northeast1)*

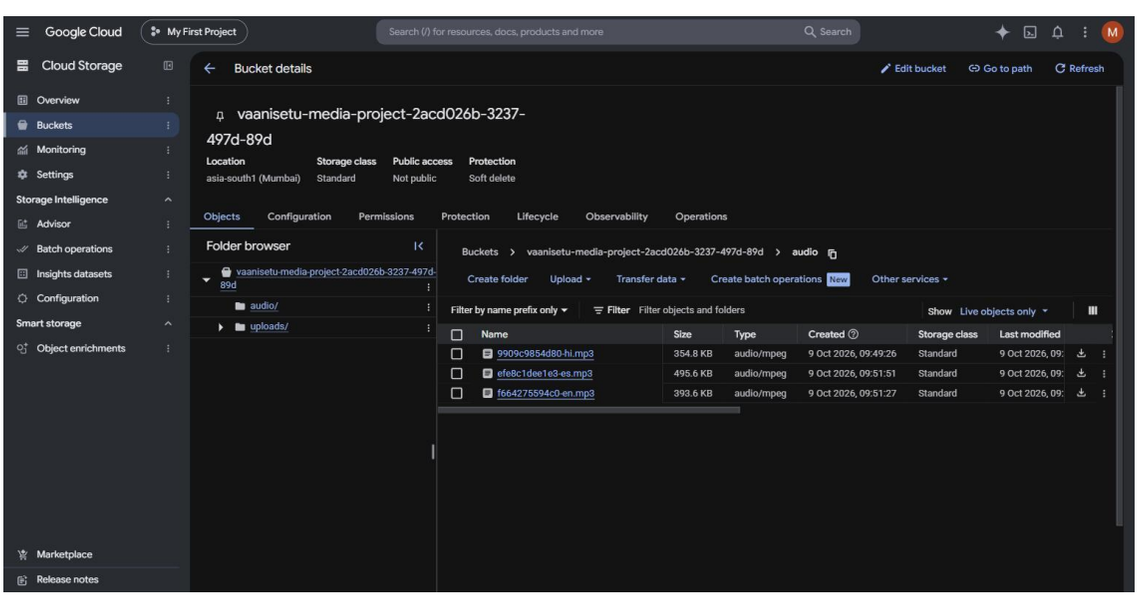



*Figure 5.8: Cloud Storage bucket with MP3 files generated by Cloud Text-to-Speech*

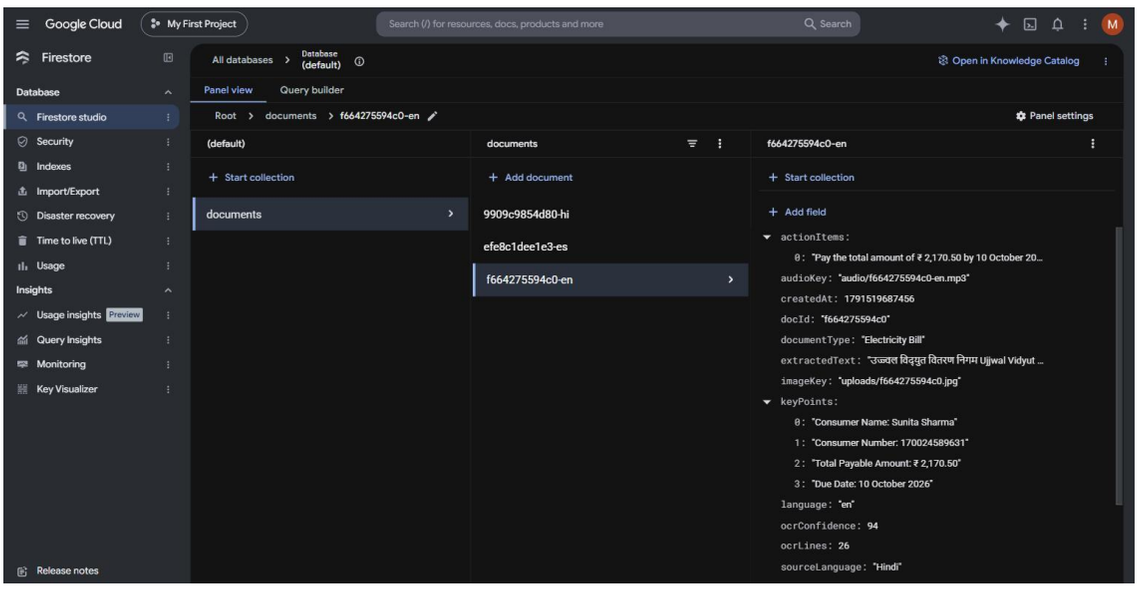



*Figure 5.9: Firestore document with the stored result*

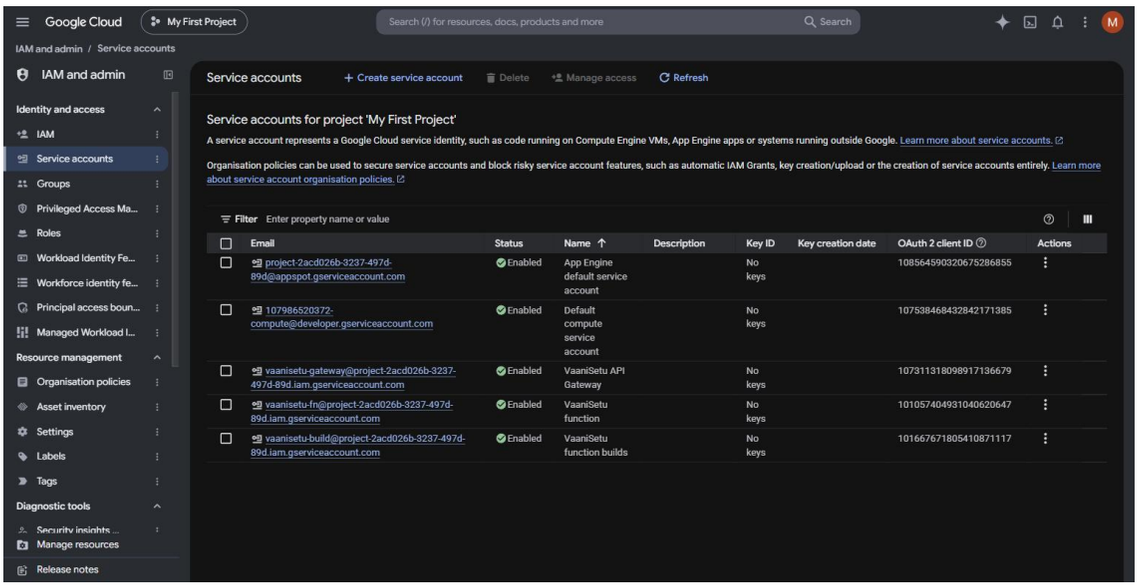



*Figure 5.10: Least-privilege IAM service accounts*

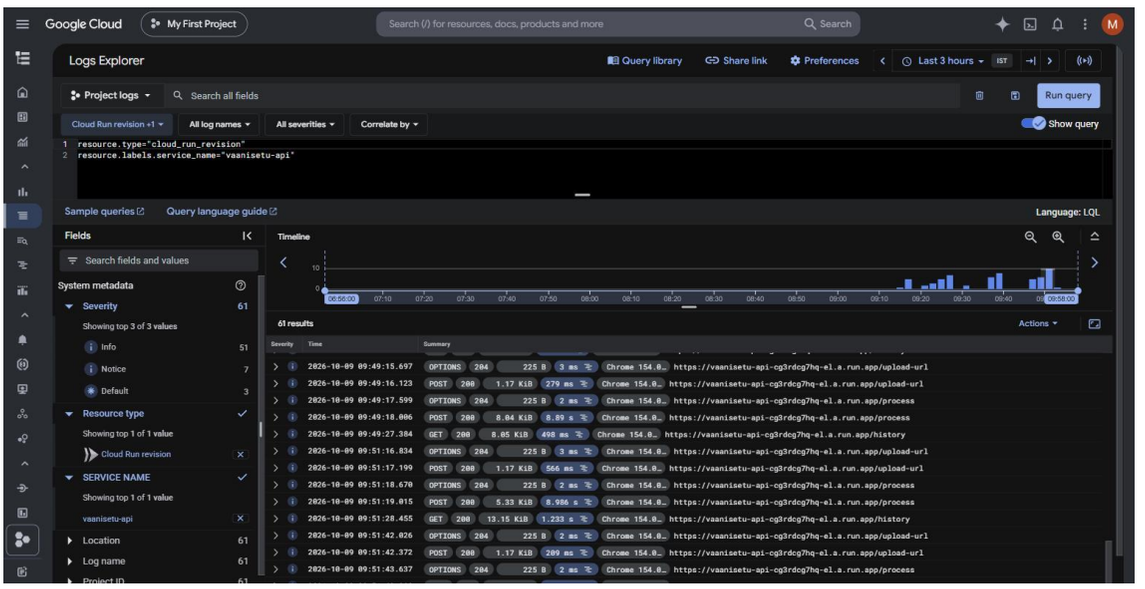



*Figure 5.11: Cloud Logging showing every API request with latency*

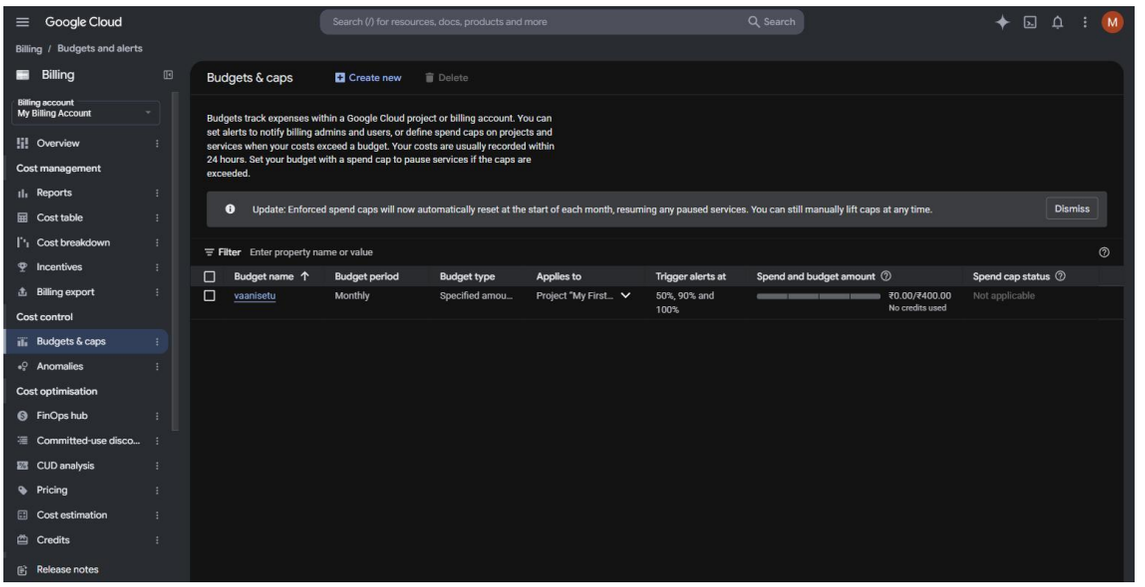



*Figure 5.12: Budget alert of 400 INR (no credits used)*

# 6. Conclusion

VaaniSetu shows how managed cloud services can be combined into a useful, socially relevant product with very little infrastructure code. A photo taken on a phone becomes a clear spoken explanation in the user's own language in about eleven seconds. In testing, Cloud Vision took about 1.5 seconds, Gemini 3.5 Flash-Lite about 3.5 seconds and Cloud Text-to-Speech about 2 seconds per document.

## 6.1 Outcomes

- A fully serverless application on Google Cloud integrating eight core services plus IAM and Cloud Logging.
- Documents in one language (for example Hindi) explained and spoken in another (for example English).
- Security by design: private buckets, short-lived signed URLs, encryption at rest and least-privilege service accounts.
- Zero idle cost: the function scales to zero and there are no servers, so the application costs nothing when nobody is using it. A budget alert guards against surprises.
- Infrastructure provisioned by a single repeatable script, with source code under version control.

## 6.2 Challenges and Learnings

- The project was first built on AWS, but the AWS account was suspended. Because the infrastructure was scripted, the whole system was rebuilt on Google Cloud service for service; the AWS version remains in the Git history.
- Gemini 3.8 Flash took 16 to 24 seconds per document. Benchmarking six Gemini models on the real prompt showed that Gemini 3.5 Flash-Lite with thinking turned off answers in about 3.5 seconds with correct output, and it is also the cheapest.
- API Gateway is not offered in Mumbai, so the gateway runs in Tokyo and routes to the function in Mumbai.
- Cloud Run functions have no private key for signing URLs, so the function signs Cloud Storage URLs through the IAM signBlob API using its own service account.

## 6.3 Future Scope

- Support multi-page PDFs using asynchronous processing with Pub/Sub and Workflows.
- Add more Indian languages (Marathi, Tamil, Bengali and others, which Cloud Text-to-Speech already supports) and a voice question-and-answer mode ("When is my next dose?").
- User accounts with Firebase Authentication so that each user's history is private.
- A WhatsApp or IVR channel so that users without smartphones can send a photo and receive a call back.

Live application: https://storage.googleapis.com/vaanisetu-web-project-2acd026b-3237-497d-89d/index.html

Source code: https://github.com/17mohak/vaanisetu